

NAME : GALLA MOHAN SAI

MODULE NO : 4

Exercise 1: Setting Up Junit

StudentMarks.java

```
public class StudentMarks {  
  
    public int totalMarks(int subject1, int subject2) {  
        return subject1 + subject2;  
    }  
}
```

StudentMarksTest.java

```
import org.junit.Test;  
import static org.junit.Assert.assertEquals;  
  
public class StudentMarksTest {  
  
    @Test  
    public void testTotalMarks() {  
  
        StudentMarks student = new StudentMarks();  
  
        int result = student.totalMarks(40, 50);  
  
        assertEquals(90, result);  
    }  
}
```

Exercise 3: Assertions in JUnit

```
import org.junit.Test;  
import static org.junit.Assert.*;  
  
public class AssertionsTest {  
  
    @Test
```

```

public void testAssertions() {

    StudentMarks student = new StudentMarks();

    assertEquals(90,
        student.totalMarks(40, 50));

    assertTrue(90 > 50);

    assertFalse(40 > 90);

    assertNull(null);

    assertNotNull(student);
}
}

```

Exercise 4: AAA Pattern + Test Fixtures + Setup and Teardown

StudentMarks.java

```

public class StudentMarks {

    public int totalMarks(int subject1, int subject2) {
        return subject1 + subject2;
    }
}

```

StudentMarksTest.java

```

import org.junit.Before;
import org.junit.After;
import org.junit.Test;

import static org.junit.Assert.*;

public class StudentMarksTest {

    private StudentMarks student;

    @Before
    public void setUp() {

```

```

        System.out.println("Setting up...");

        student = new StudentMarks();
    }

    @After
    public void tearDown() {

        System.out.println("Cleaning up...");

        student = null;
    }

    @Test
    public void testTotalMarks() {

        int subject1 = 40;
        int subject2 = 50;

        int result = student.totalMarks(subject1, subject2);

        assertEquals(90, result);
    }
}

```

Exercise 1 : Mocking and Stubbing

StudentDatabase.java

```

public class StudentDatabase {

    public String getStudentName() {

        return "Ravi";
    }
}

```

StudentService.java

```

public class StudentService {
    private StudentDatabase database;
}

```

```

    public StudentService(StudentDatabase database) {
        this.database = database;
    }
    public String fetchStudentName() {

        return database.getStudentName();
    }
}

```

StudentServiceTest.java

```

import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class StudentServiceTest {

    @Test
    public void testStudentDatabase() {

        StudentDatabase mockDatabase =
            Mockito.mock(StudentDatabase.class);

        when(mockDatabase.getStudentName())
            .thenReturn("Mock Ravi");

        StudentService service =
            new StudentService(mockDatabase);

        String result =
            service.fetchStudentName();

        assertEquals("Mock Ravi", result);
    }
}

```

Exercise 2: Verifying Interactions :

```

import static org.mockito.Mockito.*;

import org.junit.jupiter.api.Test;

```

```
import org.mockito.Mockito;

public class StudentServiceTest {

    @Test
    public void testVerifyInteraction() {

        StudentDatabase mockDatabase =
            Mockito.mock(StudentDatabase.class);

        StudentService service =
            new StudentService(mockDatabase);

        service.fetchStudentName();

        verify(mockDatabase).getStudentName();
    }
}
```

Exercise 1: Logging Error Messages and Warning Levels

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class StudentLoggingExample {

    private static final Logger logger =
        LoggerFactory.getLogger(StudentLoggingExample.class);

    public static void main(String[] args) {

        logger.error("Student record not found");

        logger.warn("Student attendance is low");
    }
}
```